

Finals Activity 1

Marlon Alen I Flores
BSCS - 3A

CS311
1980-01-01

Contents

Screen Recording	1
Scenarios	1
Code	10

Screen Recording

https://drive.google.com/file/d/1YoUenS--D1AYrLiQkV9hiKzZ_AOHbwTB/view?usp=sharing

Scenarios

Create 4 new tickets (2 hardware, 1 software, and 1 connection).

The screenshot displays a web application for managing tickets. At the top, there is a search bar with the label "Search:" and a "Search" button. Below this is a table with four columns: "Ticket Number", "Problem", "Date", and "Status". The table is currently empty. A modal form is open in the center of the screen, allowing the user to create a new ticket. The form contains the following fields:

- Ticket Number:** A text input field containing the value "20260924110732".
- Problem:** A dropdown menu with "Hardware" selected.
- Details:** A text area containing the text "Ticket 1".

At the bottom of the modal form are two buttons: "Save" and "Cancel". Below the table, there is a row of buttons: "Add", "Update", "Delete", "Refresh", "Details", "Assign", and "Approve".

Search:

Search

Ticket Number	Problem	Date	Status
Ticket Number: 20260924110744 Problem: Hardware Details: Ticket 2 Save Cancel			

Add

Update

Delete

Refresh

Details

Assign

Approve

Search:

Search

Ticket Number	Problem	Date	Status
Ticket Number: 20260924110753 Problem: Software Details: Ticket 3 Save Cancel			

Add

Update

Delete

Refresh

Details

Assign

Approve

Search:

Search

Ticket Number	Problem	Date	Status
---------------	---------	------	--------

Ticket Number:

Problem:

Details:

Ticket 4

Save

Cancel

Add

Update

Delete

Refresh

Details

Assign

Approve

Highlights the tickets you have created on instruction 1.

Search:

Search

Ticket Number	Problem	Date	Status
20260924110732	Hardware	9/24/2026 11:07:41 AM	Pending
20260924110744	Hardware	9/24/2026 11:07:51 AM	Pending
20260924110753	Software	9/24/2026 11:08:10 AM	Pending
20260924110824	Connection	9/24/2026 11:08:34 AM	Pending

Add

Update

Delete

Refresh

Details

Assign

Approve

Assign the first 3 tickets to different technical accounts.

Search: Search

Ticket Number	Problem	Status
20260924110732	Hardware	Pending
20260924110744	Hardware	Pending
20260924110753	Software	Pending
20260924110824	Configuration	Pending

Ticket Number: 20260924110732

Problem: Hardware

Description: Ticket 1

Date Created: 9/24/2026 11:07:41 AM

Assign To: tech

Cancel Save

Add Update Delete Refresh Details Assign Approve

Search: Search

Ticket Number	Problem	Status
20260924110732	Hardware	Pending
20260924110744	Hardware	Pending
20260924110753	Software	Pending
20260924110824	Configuration	Pending

Ticket Number: 20260924110744

Problem: Hardware

Description: Ticket 2

Date Created: 9/24/2026 11:07:51 AM

Assign To: tech2

Cancel Save

Add Update Delete Refresh Details Assign Approve

Search:

Search

Ticket Number	Problem	Status
20260924110732	Hardware	Pending
20260924110744	Hardware	Pending
20260924110753	Software	Pending
20260924110824	Connection	Pending

Ticket Number

20260924110753

Problem

Software

Description

Ticket 3

Date Created

9/24/2026 11:08:10 AM

Assign To

tech3

Cancel

Save

Add

Update

Delete

Refresh

Details

Assign

Approve

Show each of the tickets you have assigned on instruction3.

Search:

Search

Ticket Number	Problem	Date	Status
20260924110732	Hardware	9/24/2026 11:07:41 AM	Pending
20260924110744	Hardware	9/24/2026 11:07:51 AM	Pending
20260924110753	Software	9/24/2026 11:08:10 AM	Pending
20260924110824	Connection	9/24/2026 11:08:34 AM	Pending

Add

Update

Delete

Refresh

Details

Assign

Approve

Complete the first 2 tickets you have assigned on instruction3.

Search:

Search

Ticket Number	Problem	Status
20260924110732	Hardware	Pending

Ticket Number

20260924110732

Problem

Hardware

Details

Ticket 1

Status

Pending

Created By

admin

Date Created

9/24/2026 11:07:41 AM

Assigned To

tech

Date Assigned

9/24/2026 11:10:03 AM

Date Completed

Approved By

Date Approved

Close

Add

Update

Delete

Refresh

Details

Complete

Search:

Search

Ticket Number	Problem	Date	Status
20260924110732	Hardware	9/24/2026 11:07:41 AM	Pending

Are you sure you want to mark this ticket as complete?

Yes

No

Add

Update

Delete

Refresh

Details

Complete

Search:

Search

Ticket Number	Problem	Status
20260924110744	Hardware	Pending

Ticket Number

20260924110744

Problem

Hardware

Details

Ticket 2

Status

Pending

Created By

admin

Date Created

9/24/2026 11:07:51 AM

Assigned To

tech2

Date Assigned

9/24/2026 11:10:11 AM

Date Completed

Approved By

Date Approved

Close

Add

Update

Delete

Refresh

Details

Complete

Search:

Search

Ticket Number	Problem	Date	Status
20260924110744	Hardware	9/24/2026 11:07:51 AM	Pending

Are you sure you want to mark this ticket as complete?

Yes

No

Add

Update

Delete

Refresh

Details

Complete

Search:

Search

Ticket Number	Problem	Date	Status
20260924110744	Hardware	9/24/2026 11:07:51 AM	For_Approval

Add

Update

Delete

Refresh

Details

Complete

Show each of the tickets you have completed on instruction5.

Search:

Search

Ticket Number	Problem	Date	Status
20260924110732	Hardware	9/24/2026 11:07:41 AM	For_Approval
20260924110744	Hardware	9/24/2026 11:07:51 AM	For_Approval
20260924110753	Software	9/24/2026 11:08:10 AM	Pending
20260924110824	Connection	9/24/2026 11:08:34 AM	Pending

Add

Update

Delete

Refresh

Details

Assign

Approve

Approve the first ticket you have completed on instruction 5.

Search:

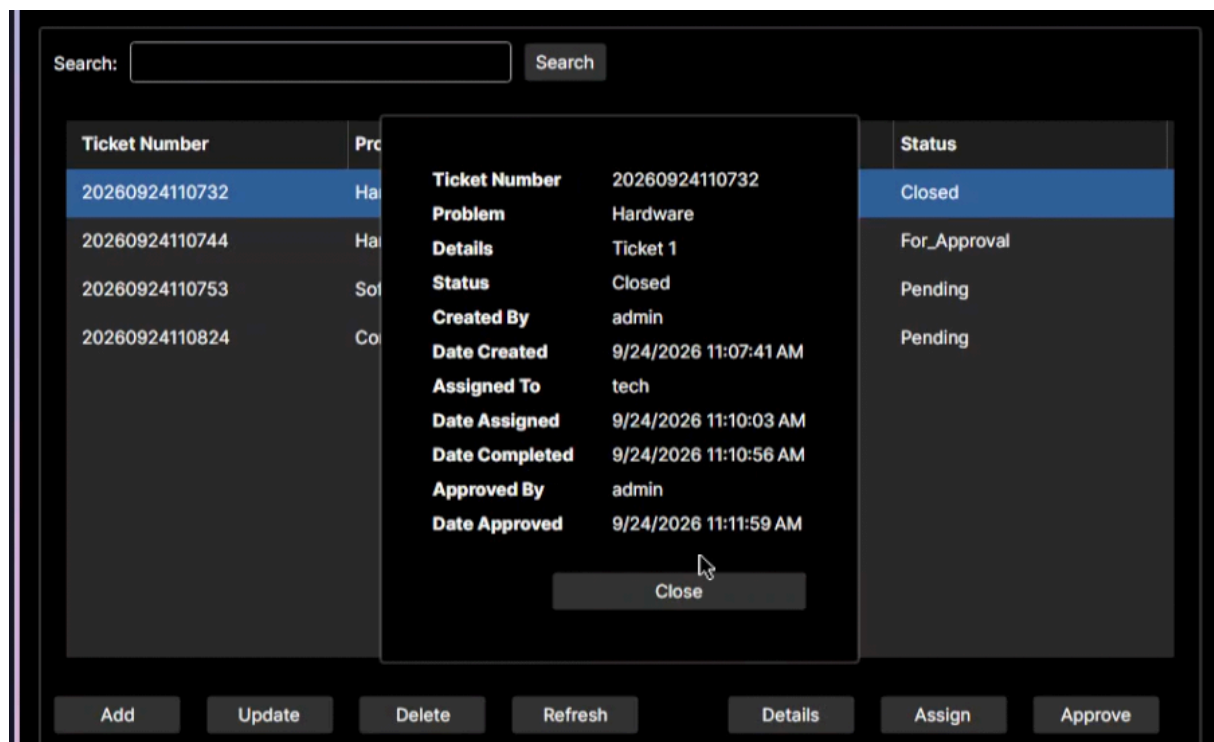
Ticket Number	Problem	Date	Status
20260924110732	Hardware	9/24/2026 11:07:41 AM	For_Approval
20260924110744	Hardware	9/24/2026 11:07:51 AM	For_Approval
20260924110753	Software	9/24/2026 11:08:10 AM	Pending
20260924110824	Connect	9/24/2026 11:08:34 AM	Pending

Are you sure you want to approve this ticket?

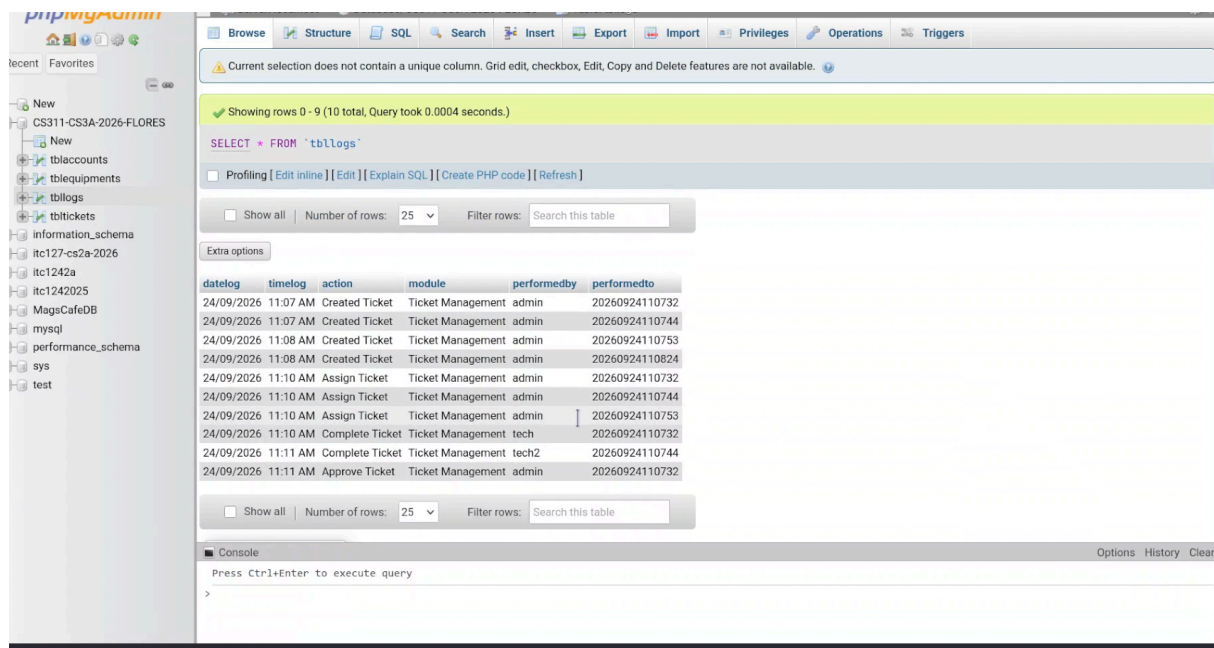
Show each of the ticket you have approved on instruction7.

Search:

Ticket Number	Problem	Date	Status
20260924110732	Hardware	9/24/2026 11:07:41 AM	Closed
20260924110744	Hardware	9/24/2026 11:07:51 AM	For_Approval
20260924110753	Software	9/24/2026 11:08:10 AM	Pending
20260924110824	Connection	9/24/2026 11:08:34 AM	Pending



Show the logs table.



Code

Full code is available at <https://github.com/Alen-Flores/CS311-2026-FLORES/tree/master/Finals/CS311FinalsActivity2-Flores>

Services

./Services/TicketService.cs

```
1 using System;
2 using System.Collections.Generic;
```

C#

```

3  using System.Data;
4  using System.Linq;
5  using CS311_CS3A_2026_Flores.Models;
6
7  namespace CS311_CS3A_2026_Flores.Services;
8
9  public interface ITicketService
10 {
11     List<Ticket> GetTickets();
12     List<Ticket> QueryTickets(string query, string username);
13     Ticket? GetTicket(int TicketNumber);
14     List<Ticket> GetTicketsByUser(string username);
15     void AddTicket(Ticket ticket);
16     void UpdateTicket(Ticket ticket);
17     void DeleteTicket(string TicketNumber);
18 }
19
20 public class TicketService : ITicketService
21 {
22
23     private DatabaseService dbService;
24     public TicketService(DatabaseService dbService)
25     {
26         this.dbService = dbService;
27     }
28
29     public Ticket? GetTicket(int TicketNumber)
30     {
31         DataTable dt = dbService.GetData($"
32         SELECT * FROM tbltickets
33         WHERE TicketNumber = '{TicketNumber}'
34         ");
35
36         DataRow? row = dt.AsEnumerable().FirstOrDefault();
37         if (row is null) return null;
38         return Ticket.FromDataRow(row);
39     }
40
41     public List<Ticket> GetTickets()
42     {
43         DataTable dt = dbService.GetData($"
44         SELECT * FROM tbltickets
45         ORDER BY DateApproved DESC
46         ");
47         return dt.AsEnumerable().Select(Ticket.FromDataRow).ToList();
48     }
49

```

```

50     public List<Ticket> GetTicketsByUser(string username)
51     {
52         DataTable dt = dbService.GetData($"
53         SELECT * FROM tbltickets
54         WHERE AssignedTo = '{username}'
55         ORDER BY DateApproved DESC
56         ");
57         return dt.AsEnumerable().Select(Ticket.FromDataRow).ToList();
58     }
59
60     public List<Ticket> QueryTickets(string query, string username)
61     {
62         DataTable dt = dbService.GetData($"
63         SELECT * FROM tbltickets
64         WHERE TicketNumber LIKE '{query}%'
65         OR Problem LIKE '{query}%'
66         OR Status LIKE '{query}%'
67         AND CreatedBy = '{username}'
68         ORDER BY DateApproved DESC
69         ");
70         return dt.AsEnumerable().Select(Ticket.FromDataRow).ToList();
71     }
72
73     private string DatetimeOrNull(DateTime? dateTime)
74     {
75         if (dateTime.HasValue)
76         {
77             return $"\"{dateTime:yyyy-MM-dd HH:mm:ss}\"";
78         }
79         else
80         {
81             return "NULL";
82         }
83     }
84
85     private string ToStringOrNull(object? ob)
86     {
87         if (ob is null)
88         {
89             return "NULL";
90         }
91         return "\"" + ob.ToString() + "\"";
92     }
93
94     public void AddTicket(Ticket ticket)
95     {
96         dbService.executeSQL($"

```

```

97     INSERT INTO tbltickets (
98         TicketNumber, Problem, Details,
99         Status, CreatedBy, DateCreated,
100         AssignedTo, DateAssigned, DateCompleted,
101         ApprovedBy, DateApproved
102     )
103     VALUES (
104         "{ticket.TicketNumber}",
105         "{ticket.Problem}",
106         "{ticket.Details}",
107         "{ticket.Status}",
108         "{ticket.CreatedBy}",
109         {DatetimeOrNull(ticket.DateCreated)},
110         {ToStringOrNull(ticket.AssignedTo)},
111         {DatetimeOrNull(ticket.DateAssigned)},
112         {DatetimeOrNull(ticket.DateCompleted)},
113         {ToStringOrNull(ticket.ApprovedBy)},
114         {DatetimeOrNull(ticket.DateApproved)}
115     )
116     "");
117 }
118
119 public void UpdateTicket(Ticket ticket)
120 {
121     dbService.executeSQL($$$$
122     UPDATE tbltickets
123     SET Problem = {ToStringOrNull(ticket.Problem)},
124         Details = {ToStringOrNull(ticket.Details)},
125         Status = {ToStringOrNull(ticket.Status)},
126         CreatedBy = {ToStringOrNull(ticket.CreatedBy)},
127         DateCreated = {DatetimeOrNull(ticket.DateCreated)},
128         AssignedTo = {ToStringOrNull(ticket.AssignedTo)},
129         DateAssigned = {DatetimeOrNull(ticket.DateAssigned)},
130         DateCompleted = {DatetimeOrNull(ticket.DateCompleted)},
131         ApprovedBy = {ToStringOrNull(ticket.ApprovedBy)},
132         DateApproved = {DatetimeOrNull(ticket.DateApproved)}
133     WHERE TicketNumber = '{ticket.TicketNumber}'
134     "");
135 }
136
137 public void DeleteTicket(string TicketNumber)
138 {
139     dbService.executeSQL($$$$
140     DELETE FROM tbltickets
141     WHERE TicketNumber = '{TicketNumber}'
142     "");
143 }

```

ViewModel

./ViewModels/Tickets/TicketsViewModel.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.ComponentModel.DataAnnotations;
4  using System.Threading.Tasks;
5  using CommunityToolkit.Mvvm.ComponentModel;
6  using CommunityToolkit.Mvvm.Input;
7  using CS311_CS3A_2026_Flores.Models;
8  using CS311_CS3A_2026_Flores.Services;
9  using CS311_CS3A_2026_Flores.Views;
10
11 namespace CS311_CS3A_2026_Flores.ViewModels;
12
13 public partial class TicketsViewModel : ObservableValidator
14 {
15
16     [ObservableProperty]
17     object? _currentControl;
18
19     [ObservableProperty]
20     private List<Ticket> _tickets;
21
22     public static ValidationResult? ValidateInRange(
23         int selectedUser,
24         ValidationContext context
25     )
26     {
27         var viewModel = (TicketAssignViewModel)context.ObjectInstance;
28
29         if (selectedUser < 0 || selectedUser > viewModel.TechnicalUsers.Count)
30             return new ValidationResult($"Select a valid user");
31
32         return ValidationResult.Success;
33     }
34
35     [ObservableProperty]
36     [CustomValidation(typeof(TicketsViewModel), nameof(ValidateInRange))]
37     private int _selectedIndex;
38
39     private readonly ITicketService _ticketService;
40     private readonly IAuthService _authService;
41     private readonly ILoggingService _loggingService;
42     private readonly IUserService _userService;
```

```

43
44     [ObservableProperty]
45     private bool _isAdmin;
46     [ObservableProperty]
47     private bool _isTechnical;
48
49     public TicketsViewModel(
50         ITicketService ticketService,
51         IAuthService authService,
52         ILoggingService loggingService,
53         IUserService userService
54     )
55     {
56         _ticketService = ticketService;
57         _authService = authService;
58         _loggingService = loggingService;
59         _userService = userService;
60         var user = authService.GetUser().GetAwaiter().GetResult();
61
62
63         IsAdmin = user.UserType == "ADMINISTRATOR";
64         IsTechnical = user.UserType == "TECHNICAL";
65
66         _tickets = IsAdmin
67             ? ticketService.GetTickets()
68             : ticketService.GetTicketsByUser(user.Username);
69
70         _authService.AccountChanged += async (e) =>
71         {
72             IsAdmin = e?.UserType == "ADMINISTRATOR";
73             IsTechnical = e?.UserType == "TECHNICAL";
74             await Refresh();
75         };
76     }
77
78     private bool CurrentIsNull()
79     {
80         return CurrentControl is null;
81     }
82
83     [RelayCommand(CanExecute = nameof(CurrentIsNull))]
84     private void Add()
85     {
86         var addTicketViewModel =
87             new AddTicketViewModel(_ticketService, _authService, _loggingService);
88         addTicketViewModel.exit += () => CurrentControl = null;
89         CurrentControl = addTicketViewModel;

```

```

90     }
91
92     [RelayCommand(CanExecute = nameof(CurrentIsNull))]
93     private void Update()
94     {
95         var target = Tickets[SelectedIndex];
96         var updateTicketViewModel =
97             new UpdateTicketViewModel(_ticketService, _authService, _loggingService,
98                 target);
99         updateTicketViewModel.exit += () => CurrentControl = null;
100         CurrentControl = updateTicketViewModel;
101     }
102     [RelayCommand(CanExecute = nameof(CurrentIsNull))]
103     private async Task Delete()
104     {
105         var target = Tickets[SelectedIndex];
106         var choice = await Dialog.Show("Are you sure you want to delete this
107             ticket?",
108             Dialog.Buttons.YesNo);
109         switch (choice)
110         {
111             case Dialog.DialogResult.Yes:
112                 _ticketService.DeleteTicket(target.TicketNumber);
113                 _loggingService.LogAction(
114                     Log.WithCurrentTimeStamp("Delete Ticket", "Ticket Management",
115                         (await _authService.GetUser()).Username, target.TicketNumber
116                     ));
117                 SelectedIndex = -1;
118                 break;
119             default:
120                 break;
121         }
122     }
123     [RelayCommand(CanExecute = nameof(CurrentIsNull))]
124     private async Task Refresh()
125     {
126         SelectedIndex = -1;
127         var user = await _authService.GetUser();
128         Tickets = IsAdmin
129             ? _ticketService.GetTickets()
130             : _ticketService.GetTicketsByUser(user.Username);
131     }
132 }
133
134 [RelayCommand]

```

```

135     private async Task Search(string query)
136     {
137         Tickets = _ticketService.QueryTickets(
138             query,
139             (await _authService.GetUser()).Username
140         );
141     }
142
143     [RelayCommand(CanExecute = nameof(CurrentIsNull))]
144     private void Details()
145     {
146         if (SelectedIndex > Tickets.Count || SelectedIndex < 0) return;
147         var target = Tickets[SelectedIndex];
148         var detailsVM = new DetailsViewModel(target);
149         detailsVM.exit += () => CurrentControl = null;
150         CurrentControl = detailsVM;
151     }
152
153     [RelayCommand(CanExecute = nameof(CurrentIsNull))]
154     private void Assign()
155     {
156         if (SelectedIndex > Tickets.Count || SelectedIndex < 0) return;
157         var target = Tickets[SelectedIndex];
158         var assignTicket = new TicketAssignViewModel(
159             target, _userService, _ticketService, _loggingService, _authService
160         );
161         assignTicket.exit += () => CurrentControl = null;
162         CurrentControl = assignTicket;
163     }
164
165     [RelayCommand(CanExecute = nameof(CurrentIsNull))]
166     private async Task Approve()
167     {
168         var user = await _authService.GetUser();
169         var target = Tickets[SelectedIndex];
170         var choice = await Dialog.Show("Are you sure you want to approve this
171             ticket?",
172             Dialog.Buttons.YesNo);
173         {
174             case Dialog.DialogResult.Yes:
175                 _ticketService.UpdateTicket(target with
176                 {
177                     Status = StatusType.Closed,
178                     DateApproved = DateTime.Now,
179                     ApprovedBy = user.Username
180                 });

```

```

181     _loggingService.LogAction(
182         Log.WithCurrentTimeStamp("Approve Ticket", "Ticket Management",
183             user.Username, target.TicketNumber
184         ));
185     break;
186     default:
187         break;
188 }
189 }
190
191 [RelayCommand(CanExecute = nameof(CurrentIsNull))]
192 private async Task Complete()
193 {
194     var user = await _authService.GetUser();
195     var target = Tickets[SelectedIndex];
196     var choice = await Dialog.Show("Are you sure you want to mark this ticket
197     as complete?",
198         Dialog.Buttons.YesNo);
199     {
200         case Dialog.DialogResult.Yes:
201             _ticketService.UpdateTicket(target with
202             {
203                 Status = StatusType.For_Approval,
204                 DateCompleted = DateTime.Now
205             });
206             _loggingService.LogAction(
207                 Log.WithCurrentTimeStamp("Complete Ticket", "Ticket Management",
208                     user.Username, target.TicketNumber
209                 ));
210             break;
211             default:
212                 break;
213         }
214     }
215 }

```

./ViewModels/Tickets/DetailsViewModel.cs

```

1  using System;
2  using CommunityToolkit.Mvvm.ComponentModel;
3  using CommunityToolkit.Mvvm.Input;
4  using CS311_CS3A_2026_Flores.Models;
5
6  namespace CS311_CS3A_2026_Flores.ViewModels;
7
8  public partial class DetailsViewModel : ObservableObject
9  {

```

C#

```

10  public event Action? exit;
11
12  [ObservableProperty]
13  Ticket _ticket;
14
15  public DetailsViewModel(Ticket ticket)
16  {
17      _ticket = ticket;
18  }
19
20  [RelayCommand]
21  private void Close()
22  {
23      exit?.Invoke();
24  }
25 }

```

./ViewModels/Tickets/TicketAssignViewModel.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.ComponentModel.DataAnnotations;
4  using System.Linq;
5  using System.Threading.Tasks;
6  using CommunityToolkit.Mvvm.ComponentModel;
7  using CommunityToolkit.Mvvm.Input;
8  using CS311_CS3A_2026_Flores.Models;
9  using CS311_CS3A_2026_Flores.Services;
10 using CS311_CS3A_2026_Flores.Views;
11
12 namespace CS311_CS3A_2026_Flores.ViewModels;
13
14 public partial class TicketAssignViewModel : ObservableValidator
15 {
16     public event Action? exit;
17
18     [ObservableProperty]
19     private Ticket _ticket;
20
21     public static ValidationResult? ValidateInRange(
22         int selectedUser,
23         ValidationContext context
24     )
25     {
26         var viewModel = (TicketAssignViewModel)context.ObjectInstance;
27
28         if (selectedUser < 0 || selectedUser > viewModel.TechnicalUsers.Count)

```

```

29     return new ValidationResult($"Select a valid user");
30
31     return ValidationResult.Success;
32 }
33
34 [ObservableProperty]
35 [CustomValidation(typeof(TicketAssignViewModel), nameof(ValidateInRange))]
36 [Required]
37 private int _selectedUser;
38
39
40 [ObservableProperty]
41 private List<string> _technicalUsers;
42
43 private readonly ITicketService _ticketService;
44 private readonly ILoggingService _loggingService;
45 private readonly IAuthService _authService;
46
47 public TicketAssignViewModel(
48     Ticket ticket,
49     IUserService userService,
50     ITicketService ticketService,
51     ILoggingService loggingService,
52     IAuthService authService
53 )
54 {
55     _ticket = ticket;
56     _ticketService = ticketService;
57     _loggingService = loggingService;
58     _authService = authService;
59     _technicalUsers = userService.GetUsers()
60         .FindAll(x => x.UserType == "TECHNICAL")
61         .Select( x => x.Username )
62         .ToList();
63 }
64
65 [RelayCommand]
66 private void Close()
67 {
68     exit?.Invoke();
69 }
70
71 [RelayCommand]
72 private async Task Save()
73 {
74     ValidateAllProperties();
75     if (HasErrors) return;

```

```
76
77     var target = Ticket with
78     {
79         AssignedTo = TechnicalUsers[SelectedUser],
80         DateAssigned = DateTime.Now
81     };
82
83     _ticketService.UpdateTicket(target);
84     _loggingService.LogAction(Log.WithCurrentTimeStamp(
85         "Assign Ticket",
86         "Ticket Management",
87         (await _authService.GetUser()).Username,
88         target.TicketNumber
89     ));
90
91     await Dialog.Show("Assigned ticket", Dialog.Buttons.Ok);
92     exit?.Invoke();
93 }
94 }
```